VDMS Frontend Integration Guide

NGO Volunteer and Donation Management System — Backend API v1.0

For the React frontend developer. Everything here was checked against the running backend; the JSON samples are real responses.

> **Single sources of truth:** this guide explains *how to wire things*. The exact schema of every request/response is in **Swagger UI** (`http://localhost:8000/docs`) and in `docs/openapi.json` . If this guide and Swagger ever disagree, Swagger is right.

Contents

1. Quick start
2. Ground rules (read this first)
3. Suggested React route map
4. Project setup: API client, auth, route guards
5. The donation flow (most important screen)
6. Screen-by-screen wiring
7. UI state rules (status → what to show)
8. Downloads: PDFs and CSV
9. Cache refresh cheat-sheet
10. Error codes catalog
11. Complete endpoint reference
12. Data shapes (TypeScript-style)
13. Demo & testing tips
14. Design decisions & limitations
15. SRS traceability
16. Integration checklist

---

1. Quick start

Run the backend (Windows, Python 3.11+)

```
cd backend
py -3.11 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env            # optional, defaults work
python -m scripts.seed_demo       # optional: fills demo data (run once on an empty DB)
uvicorn app.main:app --reload --port 8000
```

What	URL
API base	`http://localhost:8000/api/v1`
Swagger UI (try every endpoint, click **Authorize**)	`http://localhost:8000/docs`
ReDoc	`http://localhost:8000/redoc`
OpenAPI JSON	`http://localhost:8000/openapi.json` (also saved at `docs/openapi.json`)
Health check	`http://localhost:8000/health`

The database is a local SQLite file (`backend/vdms.db`) created automatically on first start. Delete it to reset.

Demo accounts (after `python -m scripts.seed_demo`)

Role	Email	Password	Notes
Admin	`admin@ngo.org`	`Admin@12345`	Created automatically on first start, even without the seed
Volunteer	`volunteer@demo.org`	`Demo@1234`	Already approved, already signed up to one event
Volunteer (pending)	`pending@demo.org`	`Demo@1234`	Use to build the "awaiting approval" screen
Donor	`donor@demo.org`	`Demo@1234`	Registered donor

Seed data includes: 2 active campaigns, 1 auto-closed campaign (goal reached), 1 draft campaign, completed + failed donations, fund-utilization entries, 3 published events + 1 draft event.

Generate typed API code (optional but recommended)

```
npx openapi-typescript ../docs/openapi.json -o src/api/schema.d.ts
```

Re-run `python -m scripts.export_openapi` (in `backend/`) whenever the backend changes to refresh `docs/openapi.json`.

Avoid CORS completely in development (recommended)

Vite proxy (`vite.config.js`):

```
export default defineConfig({
  server: { proxy: { "/api": "http://localhost:8000" } },
});
```

Then set `VITE_API_URL=` (empty) so requests go to `/api/v1/...` on the dev server. If you prefer calling `http://localhost:8000` directly, CORS is already enabled for `http://localhost:5173` and `http://localhost:3000` ; add other origins in `backend/.env` → `CORS_ORIGINS=...` .

---

2. Ground rules (read this first)

2.1 URLs, JSON, trailing slashes

- Base: `/api/v1` . All bodies are JSON (`Content-Type: application/json`).
- **Never add a trailing slash** (`/campaigns` , not `/campaigns/`). The server answers a trailing slash with a `307` redirect, and browsers frequently refuse to follow redirects on cross-origin requests that carry an `Authorization` header — so the call fails in the browser even though it works in Swagger/curl.
- Update endpoints use **`PATCH`** (send only the fields you want to change), not `PUT` .

2.2 Authentication

1. `POST /auth/login` (or a register endpoint) returns `{ access_token, token_type: "bearer", expires_in, user }` .
2. Send `Authorization: Bearer <access_token>` on every request that needs it.
3. Tokens last **8 hours** (`expires_in` = 28800 s). There is no refresh token and no logout endpoint — to log out, delete the token on the client. When it expires the API answers `401` with `code: "INVALID_TOKEN"` ; send the user to login.
4. `GET /auth/me` returns the current user, **`role`** (`admin` / `volunteer` / `donor`), and the profile ids/status. Call it on app start to restore the session.

> ⚠️ **Gotcha — stale tokens break public pages.** Several public endpoints (`/campaigns/{id}` , `/events` , `POST /donations`) *optionally* read the token. If you send an **expired** token to them you get `401` `INVALID_TOKEN` instead of the public data. The API client in §4 handles this: on `INVALID_TOKEN` it clears the token and **retries the request once without it**.

2.3 Roles at a glance

Role	Can do
(not logged in)	Browse active campaigns, browse published events, **donate as a guest**, read FAQ
`donor`	Everything above + own profile, own donation history, donate without re-typing details

Role	Can do
`volunteer`	Own profile, sign up for events (only when `status == "approved"`), tasks, history
`admin`	All `/admin/*` routes. Nothing else is needed for admin screens

- Everything under `/api/v1/admin/**` requires the admin role → otherwise `403 FORBIDDEN` (or `401` without a token).
- `/volunteers/me/**` requires role `volunteer`; `/donors/me/**` requires role `donor`.
- One account = one role. The same e-mail cannot be both a donor and a volunteer.

2.4 Money — strings, not numbers

All amounts in **responses** are strings with exactly 2 decimals: `"1500.00"`. This avoids float rounding (SRS CO-3).

```
// DISPLAY: convert only for formatting
const inr = (s) => new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR"
}).format(Number(s));
inr("1500.00") // "₹1,500.00"
```

- **Never do arithmetic on these in JS** (don't sum them yourself; the API gives totals, percentages and remaining amounts).
- In **requests** you may send a string (`"500.00"`) or a number (`500`). Rules: greater than 0, max 2 decimal places, otherwise `422` with `field: "amount"`.
- Keep the amount input as a *string* in your form state and send it as-is.
- Currency is always INR (`GET /meta/config` → `currency`).

2.5 Dates and times

Kind	Format	Example	Fields
Date only	`YYYY-MM-DD`	`"2026-10-23"`	`start_date`, `end_date`, `utilization_date`
Date-time	ISO-8601 **UTC** with `Z`	`"2026-10-19T05:23:57.655088Z"`	`event_date`, `created_at`, `donation_date`, …

- The SRS shows dates as `DD/MM/YYYY` — that is a **display** format. Convert in the UI (helpers below). The API always uses ISO.
- Sending `event_date`: use `new Date(inputValue).toISOString()` (explicit UTC). If you ever send a datetime without a timezone, the server assumes **Asia/Kolkata**.
- "Today" for campaign deadlines is evaluated in Asia/Kolkata. A campaign with `end_date = 2026-10-23` closes automatically when that day is over.

```
export const fmtDate = (d) => (d ? new Date(d + "T00:00:00").toLocaleDateString("en-GB") : ""); //
23/10/2026
export const fmtDateTime = (iso) => new Date(iso).toLocaleString("en-IN", { dateStyle: "medium",
timeStyle: "short" });
```

2.6 Pagination

List endpoints that can grow take `?page=1&page_size=20` (`page_size` max 100) and return:

```
{ "items": [ ... ], "total": 3, "page": 1, "page_size": 1, "pages": 3 }
```

`pages` is `0` when `total` is `0`. Endpoints that return a plain array (`/volunteers/me/tasks` , `/volunteers/me/history` , `/admin/events/{id}/registrations` , `/admin/campaigns/{id}/utilizations` , `/admin/donors/duplicates` , `/meta/faq` , `/campaigns/categories`) are **not** paginated.

2.7 Errors — one shape everywhere

```
{ "detail": "Human readable message", "code": "MACHINE_CODE", "errors": [ { "field": "amount",
"message": "..." } ] }
```

- `detail` — safe to show to the user.
- `code` — branch your logic on this, never on `detail` text. Full list in §10.
- `errors` — present for form problems; `field` is the JSON field name (nested uses dots, e.g. `donor.email`). Use it for **inline validation messages** (SRS UI-4).

HTTP	Meaning	Typical UI reaction
400	A business rule on a field was violated	Show inline under `errors[].field`
401	Not logged in / bad or expired token	Go to login
403	Logged in but not allowed	Show "not allowed" or the specific message
404	Not found	404 page / "no longer available"
409	Conflict with current state (already registered, event full, campaign closed…)	Toast with `detail` , refetch the data
422	Request failed validation	Inline messages from `errors[]`
500	Server bug	Generic error message

2.8 Enum values (lower-case strings)

Also available live from `GET /meta/config` (handy for dropdowns).

Enum	Values
role	`admin` , `volunteer` , `donor`
volunteer status	`pending` , `approved` , `deactivated`
campaign status	`draft` , `active` , `closed` (`closed_reason` : `goal_reached` , `end_date_passed` , `manual`)
donation status	`pending` , `completed` , `failed`

Enum	Values
payment method	`card`, `upi`, `net_banking`
event status	`draft`, `published`, `cancelled`, `completed`
attendance status	`registered`, `attended`, `no_show`

2.9 Input rules the server enforces (mirror them in forms)

- **Email** — valid address; stored lower-case.
- **Password** — at least 8 characters, at least one letter and one digit, at most 72 bytes.
- **Phone** — 7–15 digits, optional leading `+`; spaces, dashes and brackets are stripped by the server (`"+91 98765-43210"` → `"+919876543210"`).
- **PAN / tax-ID** (optional on donations) — 5–20 letters/digits; stored upper-case.
- Campaign `title` ≤ 100 chars; `category` ≤ 60; event `title` ≤ 150; `location` ≤ 255; `volunteers_required` 1–10000.
- Campaign: `end_date` must be later than `start_date`; `goal_amount` > 0.

---

3. Suggested React route map

Public routes need no login. A route marked with a role must be wrapped in the route guard from §4.

Route	Who	Page	Main API calls
`/`	public	Home: hero + featured campaigns + upcoming events	`GET /campaigns?sort=ending_soon&page_size=6`, `GET /events?page_size=3`
`/campaigns`	public	Active campaigns list with filters	`GET /campaigns`, `GET /campaigns/categories`
`/campaigns/:id`	public	Campaign detail + **Donate** button	`GET /campaigns/{id}`
`/campaigns/:id/donate`	public	Donation flow (3 screens inside one page)	see §5
`/donations/:id/receipt`	public / donor	Receipt page	`GET /donations/{id}/receipt`
`/events`	public	Published events (volunteers see their sign-up state)	`GET /events`
`/events/:id`	public	Event detail + **Sign up** (volunteer)	`GET /events/{id}`, `POST/DELETE /events/{id}/signup`
`/help`	public	FAQ	`GET /meta/faq`
`/login`	public	Login	`POST /auth/login`
`/register/volunteer`	public	Volunteer registration	`POST /auth/register/volunteer`

Route	Who	Page	Main API calls
`/register/donor`	public	Donor registration	`POST /auth/register/donor`
`/donor`	donor	Dashboard	`GET /donors/me/dashboard`
`/donor/donations`	donor	Donation history	`GET /donors/me/donations`
`/donor/profile`	donor	Profile edit	`GET/PATCH /donors/me`
`/volunteer`	volunteer	Dashboard	`GET /volunteers/me/dashboard`
`/volunteer/tasks`	volunteer	My upcoming tasks	`GET /volunteers/me/tasks`
`/volunteer/history`	volunteer	Participation history	`GET /volunteers/me/history`
`/volunteer/profile`	volunteer	Profile edit	`GET/PATCH /volunteers/me`
`/admin`	admin	Dashboard (funds totals + counts)	`GET /admin/dashboard`
`/admin/campaigns`	admin	All campaigns (any status)	`GET /admin/campaigns`
`/admin/campaigns/new`	admin	Create campaign	`POST /admin/campaigns`
`/admin/campaigns/:id`	admin	Detail with tabs: **Overview/Edit**, **Donations**, **Fund utilization & report**	see §6.5
`/admin/events`	admin	All events	`GET /admin/events`
`/admin/events/new`	admin	Create event	`POST /admin/events`
`/admin/events/:id`	admin	Edit, publish/cancel, **roster, tasks, attendance**	see §6.6
`/admin/volunteers`	admin	Approve / deactivate volunteers	`GET /admin/volunteers`
`/admin/volunteers/:id`	admin	Volunteer profile + assignment history	`GET /admin/volunteers/{id}`
`/admin/donors`	admin	Search donors	`GET /admin/donors?q=`
`/admin/donors/duplicates`	admin	Duplicate donor review + merge	`GET /admin/donors/duplicates`, `POST /admin/donors/merge`
`/admin/donors/:id`	admin	Donor record	`GET /admin/donors/{id}`
`/admin/donations`	admin	All donations with filters	`GET /admin/donations`

Route	Who	Page	Main API calls
`/admin/audit-log`	admin	Audit trail	`GET /admin/audit-log`
`/admin/email-log`	admin	E-mails sent / skipped	`GET /admin/email-log`
`/admin/staff`	admin	Add another admin	`POST /admin/admins`

After login, redirect by role: admin → `/admin`, volunteer → `/volunteer`, donor → `/donor`. Put `/admin/donors/duplicates` **before** `/admin/donors/:id` in your router config only if you use a matcher that cares about order (React Router v6 ranks static segments higher, so it is fine).

---

4. Project setup: API client, auth, route guards

Plain JavaScript (React + Vite). Strip the JSX/JS as needed for your setup; TypeScript users can type everything from `schema.d.ts`.

4.1 `src/api/client.js`

```
const BASE = `${import.meta.env.VITE_API_URL ?? "http://localhost:8000"}/api/v1`;
const TOKEN_KEY = "vdms_token";

export const tokenStore = {
  get: () => localStorage.getItem(TOKEN_KEY),
  set: (t) => localStorage.setItem(TOKEN_KEY, t),
  clear: () => localStorage.removeItem(TOKEN_KEY),
};

export class ApiError extends Error {
  constructor(status, body) {
    super(body?.detail ?? `Request failed (${status})`);
    this.status = status;
    this.code = body?.code ?? `HTTP_${status}`;
    this.errors = body?.errors ?? [];
  }
  /** { amount: "message", "donor.email": "message" } — feed this to your form library */
  get fieldErrors() {
    return Object.fromEntries(this.errors.filter((e) => e.field).map((e) => [e.field, e.message]));
  }
}

let onSessionExpired = () => {};
export const setSessionExpiredHandler = (fn) => { onSessionExpired = fn; };

const readJson = async (res) => { try { return await res.json(); } catch { return null; } };

/**
 * api("/campaigns", { params: { page: 2 } })
 * api("/auth/login", { method: "POST", body: { email, password } })
 * api(path, { raw: true })  -> returns the fetch Response (for PDFs/CSV)
 */
export async function api(path, { method = "GET", body, params, headers = {}, raw = false } = {},
_retry = true) {
  const url = new URL(BASE + path, window.location.origin);
  for (const [k, v] of Object.entries(params ?? {})) {
    if (v !== undefined && v !== null && v !== "") url.searchParams.set(k, v);
  }
  const h = { ...headers };
  if (body !== undefined) h["Content-Type"] = "application/json";
  const token = tokenStore.get();
  if (token) h.Authorization = `Bearer ${token}`;

  const res = await fetch(url, { method, headers: h, body: body !== undefined ? JSON.stringify(body)
: undefined });

  if (!res.ok) {
    const err = new ApiError(res.status, await readJson(res));
    if (err.code === "INVALID_TOKEN" && token && _retry) {
      tokenStore.clear();            // expired / bad token
      onSessionExpired();            // AuthProvider sets user = null
      return api(path, { method, body, params, headers, raw }, false); // retry as anonymous (public
pages keep working)
```

```
    }
    throw err;
  }
  if (raw) return res;
  return res.status === 204 ? null : res.json();
}

/** Download a PDF/CSV that needs the Authorization header (see §8) */
export async function download(path, params, fallbackName = "download") {
  const res = await api(path, { params, raw: true });
  const blob = await res.blob();
  const name = /filename="?([^";]+)"?/.exec(res.headers.get("Content-Disposition") ?? "")?.[1] ??
fallbackName;
  const a = Object.assign(document.createElement("a"), { href: URL.createObjectURL(blob), download:
name });
  document.body.appendChild(a); a.click(); a.remove();
  URL.revokeObjectURL(a.href);
}
```

4.2 `src/auth/AuthContext.jsx`

```
import { createContext, useContext, useEffect, useState } from "react";
import { api, tokenStore, setSessionExpiredHandler } from "../api/client";

const AuthCtx = createContext(null);
export const useAuth = () => useContext(AuthCtx);
export const homeFor = (role) => ({ admin: "/admin", volunteer: "/volunteer", donor: "/donor" })
[role] ?? "/";

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(!!tokenStore.get());

  useEffect(() => {
    setSessionExpiredHandler(() => setUser(null));
    if (!tokenStore.get()) return;
    api("/auth/me").then(setUser).catch(() => tokenStore.clear()).finally(() => setLoading(false));
  }, []);

  const accept = (res) => { tokenStore.set(res.access_token); setUser(res.user); return res.user; };

  const value = {
    user, loading,
    login: async (email, password) => accept(await api("/auth/login", { method: "POST", body: {
email, password } })),
    registerVolunteer: async (body) => accept(await api("/auth/register/volunteer", { method:
"POST", body })),
    registerDonor: async (body) => accept(await api("/auth/register/donor", { method: "POST", body
})),
    logout: () => { tokenStore.clear(); setUser(null); },
    /** call after anything that changes the user (e.g. volunteer gets approved) */
    refreshUser: async () => setUser(await api("/auth/me")),
  };
  return <AuthCtx.Provider value={value}>{children}</AuthCtx.Provider>;
}
```

4.3 Route guard

```
import { Navigate, useLocation } from "react-router-dom";
import { useAuth, homeFor } from "./AuthContext";

export function RequireRole({ roles, children }) {
  const { user, loading } = useAuth();
  const location = useLocation();
  if (loading) return <FullPageSpinner />;
  if (!user) return <Navigate to="/login" state={{ from: location }} replace />;
  if (!roles.includes(user.role)) return <Navigate to={homeFor(user.role)} replace />;
  return children;
}
```

```jsx
// router (React Router v6)
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/campaigns" element={<Campaigns />} />
  <Route path="/campaigns/:id" element={<CampaignDetail />} />
  <Route path="/campaigns/:id/donate" element={<DonateFlow />} />   {/* public: guests can donate
*/}
  <Route path="/login" element={<Login />} />

  <Route path="/admin/*" element={<RequireRole roles={["admin"]}><AdminLayout /></RequireRole>} />
  <Route path="/volunteer/*" element={<RequireRole roles={["volunteer"]}><VolunteerLayout />
</RequireRole>} />
  <Route path="/donor/*" element={<RequireRole roles={["donor"]}><DonorLayout /></RequireRole>} />
</Routes>
```

After a successful login, navigate to `location.state?.from ?? homeFor(user.role)`.

4.4 Showing server validation errors on forms (UI-4)

```jsx
// react-hook-form example
try {
  await api("/admin/campaigns", { method: "POST", body: values });
} catch (e) {
  if (e instanceof ApiError && e.errors.length) {
    for (const [field, message] of Object.entries(e.fieldErrors)) setError(field, { message });
  } else {
    toast.error(e.message);                    // 409 etc. -> toast with detail
  }
}
```

Field names match the request body (`goal_amount`, `end_date`, `donor.email` ...), so they map straight onto form fields.

4.5 Data fetching (TanStack Query suggested)

```jsx
const { data } = useQuery({
  queryKey: ["campaigns", { category, page }],
  queryFn: () => api("/campaigns", { params: { category, page, page_size: 12 } }),
});
const donate = useMutation({
  mutationFn: (body) => api("/donations", { method: "POST", body }),
});
```

See §9 for what to invalidate after each mutation.

---

5. The donation flow (most important screen)

SRS usability target: **a first-time donor completes a donation in ≤ 4 steps**. The API is built for exactly this: *form → confirm → result/receipt*.

```
[1 Form]     amount, payment method, (donor details if guest)
    |  POST /donations
    v
[2 Confirm]  show the `confirmation` summary + "Pay" / "Cancel"
    |  POST /donations/{id}/confirm
    +--> status "completed"  -->  [3 Receipt] (on screen + PDF)
    +--> status "failed"     -->  Retry (POST /donations/{id}/retry, then back to [2]) or leave
```

5.1 Step 1 — start the donation

`POST /donations` (no login needed)

Guest body:

```
{
  "campaign_id": 1,
  "amount": "250.00",
  "payment_method": "upi",
  "donor": { "name": "Sam", "email": "sam@example.com", "phone_number": "+919876543210" },
  "pan_tax_id": "ABCDE1234F"
}
```

- **Logged-in donor**: omit `donor` — the profile is used automatically (server ignores `donor` if present).
- **Logged-in admin/volunteer** donating: they are treated as a guest; `donor` is required.
- `phone_number` and `pan_tax_id` are optional.

Response `201`:

```
{
  "donation": {
    "donation_id": 9, "campaign_id": 1, "campaign_title": "School Supplies Drive",
    "donor_id": 8, "donor_name": "Sam", "amount": "250.00", "payment_method": "upi",
    "status": "pending", "receipt_number": null,
    "donation_date": "2026-10-14T05:24:39.659906Z", "completed_at": null,
    "failure_reason": null, "retry_of_id": null
  },
  "confirmation": { "campaign_title": "School Supplies Drive", "amount": "250.00", "payment_method":
"upi", "donor_name": "Sam" },
  "access_token": "gfLs2eJDgnnIwZIteze_B--tancOWahnUqN_qn6Oeco"
}
```

Keep `access_token` — it is only returned now. It authorises confirm / cancel / retry / receipt for this donation (essential for guests). Store it in component state and in `sessionStorage` under `donation:<id>` so a page refresh on step 2 doesn't lose it.

Errors to handle here: `422` field errors (amount, email…), `400 DONATION_LIMIT_EXCEEDED` (limit in `GET /meta/config` → `max_single_donation`), `400 DONOR_DETAILS_REQUIRED`, `409 CAMPAIGN_NOT_ACTIVE` ("This campaign just closed"), `404` unknown campaign.

5.2 Step 2 — confirm screen

Show `confirmation` (campaign, amount, payment method, donor name) with **Pay** and **Cancel** buttons. Nothing has been charged and the campaign total has not changed yet.

`POST /donations/{donation_id}/confirm` with header `X-Donation-Token: <access_token>`. Body is optional. In demo builds you can force the outcome: `{ "simulate_outcome": "success" }` or `"failure"` (see §13).

Always **disable the Pay button while the request is in flight** — a double submit returns `409 DONATION_NOT_PENDING` (it is safe, nothing is charged twice, but it looks like an error).

Response `200` is the donation with its new status:

- **Success** → `status: "completed"`, `receipt_number: "RCP-20261014-000009"` → go to step 3.
- **Failure** → `status: "failed"`, `failure_reason: "Payment via card was declined (simulated)"` → show the failure screen. HTTP is still `200`; **check `status`**.
- `409 CAMPAIGN_NOT_ACTIVE` → the campaign closed between steps 1 and 2; the donation is marked failed. Show a message and link back to the campaigns list.

5.3 Failure screen — retry or cancel

Button	Call	Result
Retry	`POST /donations/{id}/retry` (header `X-Donation-Token: <old token>`)	`201` — same shape as step 1: a **new** pending donation with a **new** `access_token`. Replace your stored donation + token and go back to step 2
Cancel / back	nothing needed (the failed attempt is already recorded)	navigate away

To abandon a donation that is still `pending` (user left the confirm screen deliberately): `POST /donations/{id}/cancel` → it becomes `failed` with `failure_reason: "Cancelled by donor"`.

5.4 Step 3 — success + receipt

`GET /donations/{id}/receipt` (header `X-Donation-Token`, or JWT for the owner):

```
{
  "receipt_number": "RCP-20261014-000009", "ngo_name": "Helping Hands NGO",
  "donor_name": "Sam", "donor_email": "sam@example.com", "pan_tax_id": null,
  "amount": "250.00", "currency": "INR", "campaign_title": "School Supplies Drive",
  "payment_method": "upi", "payment_reference": "SIM-908215B15271",
  "donation_date": "2026-10-14T05:24:39.856976Z"
}
```


Show it on screen, plus a **Download PDF** button — see §8. The server also e-mails the receipt to the donor's e-mail when SMTP is configured (otherwise it only logs it; visible in the admin e-mail log). Tell the user "A copy has been e-mailed" only if you know SMTP is on; safer wording: "Your receipt is also sent to *email*".

Receipt is only available for `completed` donations (`409 RECEIPT_NOT_AVAILABLE` otherwise).

5.5 Reference implementation

```
function DonateFlow() {
  const { id } = useParams();                        // campaign id
  const { user } = useAuth();
  const isDonor = user?.role === "donor";
  const [step, setStep] = useState("form");          // form | confirm | done | failed
  const [init, setInit] = useState(null);            // response of POST /donations
  const [donation, setDonation] = useState(null);    // latest donation object (after confirm)
  const [busy, setBusy] = useState(false);

  const tokenHeader = () => ({ "X-Donation-Token": init.access_token });
  const did = () => init.donation.donation_id;

  async function start(v) {                           // v: { amount, method, name, email, phone, pan
}
    const body = {
      campaign_id: Number(id), amount: v.amount, payment_method: v.method,
      pan_tax_id: v.pan || undefined,
      ...(isDonor ? {} : { donor: { name: v.name, email: v.email, phone_number: v.phone || undefined
} }),
    };
    const res = await api("/donations", { method: "POST", body });   // may throw ApiError -> inline
errors
    sessionStorage.setItem(`donation:${res.donation.donation_id}`, res.access_token);
    setInit(res); setStep("confirm");
  }

  async function pay() {
    setBusy(true);
    try {
      const d = await api(`/donations/${did()}/confirm`, { method: "POST", headers: tokenHeader()
});
      setDonation(d);
      setStep(d.status === "completed" ? "done" : "failed");
    } catch (e) {
      if (e.code === "CAMPAIGN_NOT_ACTIVE") setStep("closed");
      else throw e;
    } finally { setBusy(false); }
  }

  async function retry() {
    const res = await api(`/donations/${did()}/retry`, { method: "POST", headers: tokenHeader() });
    setInit(res); setStep("confirm");                 // NEW donation id + NEW token
  }

  // render <DonationForm/>, <ConfirmCard summary={init.confirmation}/>, <Success donation=
{donation}/>, <Failed reason={donation.failure_reason}/>
}
```

6. Screen-by-screen wiring

Notation: **GET** `/path` → what to render. "Auth" says what token is needed.

6.1 Public pages

Campaign list `/campaigns` — Auth: none * `GET /campaigns?page=1&page_size=12&category=education&q=books&sort=newest` * `sort`: `newest` (default), `ending_soon`, `most_funded`; `category` exact match (case-insensitive); `q` searches title + description. * Category filter options: `GET /campaigns/categories` → `["education", "healthcare"]`. * Card fields: `title`, `category`, `goal_amount`, `amount_raised`, `progress_percent` (0–100, use for the progress bar), `amount_remaining`, `end_date` (show "N days left" computed from it). * Only **active** campaigns appear here. The list is always fresh: campaigns past their end date/goal never show up.

Campaign detail `/campaigns/:id` — Auth: none * `GET /campaigns/{id}` → `CampaignOut`. Show a **Donate** button only when `status === "active"`. If `closed`, show a "Campaign closed" badge with `closed_reason` text (`goal_reached` → "Goal reached 🎉", `end_date_passed` → "Ended", `manual` → "Closed"). * Drafts return `404` to everyone but admins.

Events `/events`, `/events/:id` — Auth: optional (send the token if logged in) * `GET /events?page=1&q=camp&include_past=false` — published, upcoming events. * Each event has `registered_count`, `slots_remaining`, `is_full`, and `my_registration` (non-null only when the logged-in volunteer is already signed up → show "You're signed up" + **Withdraw**). * Button logic is in §7.

FAQ `/help` — `GET /meta/faq` → `[ { topic, question, answer } ]`, group by `topic`.

6.2 Auth screens

Login — `POST /auth/login` `{ email, password }` → store token, redirect by role. * `401 INVALID_CREDENTIALS` ("Incorrect e-mail or password") — show as a form-level error (the API deliberately doesn't say which part is wrong).

Register volunteer — `POST /auth/register/volunteer`

```
{ "name": "Asha Rao", "email": "asha@example.com", "password": "Passw0rdX",
  "phone_number": "+91 98765 43210", "areas_of_interest": ["teaching", "logistics"], "availability":
"Weekends" }
```

- `areas_of_interest`: a multi-select / chips (≥ 1). Suggestions: `GET /meta/config` → `suggested_volunteer_interests` (users may type their own).
- Returns `AuthResponse` (already logged in) with `user.volunteer_status: "pending"` → redirect to `/volunteer` and show the **"Awaiting admin approval"** banner.
- `409 EMAIL_ALREADY_REGISTERED` / `409 PHONE_ALREADY_REGISTERED` come with `errors[].field` (`email` / `phone_number`) → inline.

Register donor — `POST /auth/register/donor` `{ name, email, password, phone_number? }` → logged in, redirect to `/donor`.

Change password — `POST /auth/change-password` `{ current_password, new_password }` → `400 WRONG_PASSWORD` (field `current_password`).

6.3 Donor area (role `donor`)

- **Dashboard** `GET /donors/me/dashboard` → `json { "donor": { "donor_id": 1, "name": "...", "email": "...", "total_donated": "0.00", "donation_count": 0, ... }, "recent_donations": [ DonationOut ], "featured_campaigns": [ CampaignOut ] }` `total_donated` / `donation_count` count **completed** donations only.
- **History** `GET /donors/me/donations?page=1&status=completed` — `status` optional (`pending` / `completed` / `failed`; omit for all). Each row has `receipt_number` (null unless completed) → link "Receipt" only when completed.
- **Profile** `GET /donors/me` (includes totals), `PATCH /donors/me` `{ name?, phone_number? }`. E-mail is read-only (it is the login id).
- Donors can only ever see their own donations (other donors' → `403`).

6.4 Volunteer area (role `volunteer`)

- **Dashboard** `GET /volunteers/me/dashboard` → `{ volunteer, upcoming_assignments: [RegistrationOut], events_attended, open_events: [EventOut] }` (`open_events` = next 6 published events, each with `my_registration`).
- **My tasks** `GET /volunteers/me/tasks` → upcoming, non-cancelled events the volunteer signed up for, each with `task` (may be `null` until an admin assigns one → show "Task not assigned yet"). `json { "registration_id": 1, "event": { "event_id": 1, "title": "Food Distribution Camp", "event_date": "2026-10-20T05:23:37Z", "location": "...", "status": "published" }, "task": "registration desk", "attendance_status": "registered", "registered_at": "..." }`
- **History** `GET /volunteers/me/history` → every sign-up, newest first, with `attendance_status` (`registered` / `attended` / `no_show`).
- **Sign up** `POST /events/{id}/signup` → `201 RegistrationOut`. **Withdraw** `DELETE /events/{id}/signup` → `200 {"detail": "..."}` (only before the event starts).
- **Profile** `GET /volunteers/me`, `PATCH /volunteers/me` `{ name?, phone_number?, areas_of_interest?, availability? }`.
- **Approval banner**: read `volunteer.status` (from `/volunteers/me`, `/auth/me → user.volunteer_status`, or the dashboard):
- `pending` → "Your account is awaiting approval. You can sign up for events once an admin approves it."
- `deactivated` → "Your account is deactivated. Contact the NGO."
- Status changes when an admin acts, so call `refreshUser()` when the volunteer dashboard mounts.

6.5 Admin — campaigns & funds

Campaign list `GET /admin/campaigns?status=draft&q=flood&page=1` (`status` optional).

Create `POST /admin/campaigns` →

```
{ "title": "Flood Relief", "description": "…", "category": "disaster relief",
  "goal_amount": "50000.00", "start_date": "2026-10-14", "end_date": "2026-11-14" }
```

Response `201` has `status: "draft"`. Validation → `422` with `errors[].field` (`end_date`, `goal_amount`, …).

Lifecycle buttons (show per status — see §7): | Button | Call | Notes | |---|---|---| | Publish | `POST /admin/campaigns/{id}/publish` | draft → active. `400 INVALID_DATES` if `end_date` already passed | | Close early | `POST /admin/campaigns/{id}/close` | active → closed (`closed_reason: "manual"`). Ask for confirmation | | Save changes | `PATCH /admin/campaigns/{id}` | see rule below |

Edit rule (SRS Campaign.Edit): while `draft` every field is editable. While `active` only **`description`** and **`end_date`** are — disable the other inputs (title, category, goal, start date). If you send a locked field with a *different* value you get `409 CAMPAIGN_FIELD_LOCKED`; sending it unchanged is fine. `closed` campaigns cannot be edited (`409 CAMPAIGN_CLOSED`).

Donations tab `GET /admin/donations?campaign_id={id}&status=completed&page=1`.

Fund utilization tab * Record: `POST /admin/campaigns/{id}/utilizations` `json { "category": "supplies", "amount": "8000.00", "description": "Notebooks (500 units)", "reference_number": "INV-2026-001", "utilization_date": "2026-10-14" }` `utilization_date` optional (defaults to today). Allowed for **active and closed** campaigns, not drafts (`409 CAMPAIGN_NOT_STARTED`). If the total spent would exceed the amount raised → `400 UTILIZATION_EXCEEDS_RAISED` with `errors: [{ field: "amount", message: "Maximum allowed is 6750.50" }]` → show inline. Tip: pre-fill the max from the report's `remaining_balance`. * List entries: `GET /admin/campaigns/{id}/utilizations` (array, newest first). * Report: `GET /admin/campaigns/{id}/fund-report` → `json { "campaign_id": 1, "campaign_title": "School Supplies Drive", "campaign_status": "active", "total_raised": "16250.50", "total_utilized": "9500.00", "remaining_balance": "6750.50", "breakdown": [{ "category": "supplies", "amount": "8000.00", "entries": 1, "percent_of_utilized": 84.2 }], "entries": [UtilizationOut], "generated_at": "…" }` Use `breakdown` for a pie/bar chart (`percent_of_utilized` is already computed). * Export: **Download PDF / CSV** → `GET /admin/campaigns/{id}/fund-report/export?format=pdf|csv` (needs the auth header → use `download()` from §4.1; see §8). * Category input: free text; suggestions from `GET /meta/config` → `suggested_utilization_categories`.

Dashboard `GET /admin/dashboard` →

```
{ "total_raised": "51500.50", "total_utilized": "21500.00", "total_remaining": "30000.50",
  "campaigns": { "draft": 1, "active": 3, "closed": 1 },
  "total_donors": 8, "pending_volunteers": 1, "approved_volunteers": 16, "upcoming_events": 4,
  "recent_donations": [ DonationOut ] }
```

Use `pending_volunteers` for a "N volunteers waiting for approval" call-to-action linking to `/admin/volunteers?status=pending`.

6.6 Admin — events

Screen / button	Call
List	`GET /admin/events?status=published&q=camp&page=1`
Create	`POST /admin/events` `{ title, description, event_date, location, volunteers_required }` → `201` (status `draft`)

Screen / button	Call
Edit	`PATCH /admin/events/{id}` (partial). `400 SLOTS_BELOW_REGISTERED` if you lower `volunteers_required` below current sign-ups
Publish	`POST /admin/events/{id}/publish` (draft → published, visible to volunteers)
Cancel	`POST /admin/events/{id}/cancel` — confirm dialog: *"All N signed-up volunteers will be e-mailed."* Use `registered_count`.
Roster	`GET /admin/events/{id}/registrations` → `[ { registration_id, volunteer: {volunteer_id, name, email, phone_number}, task, attendance_status, registered_at } ]`
Assign task	`PATCH /admin/events/{id}/registrations/{registrationId}` `{ "task": "registration desk" }` (send `""` to clear)
Mark attendance (one)	same PATCH with `{ "attendance_status": "attended" }`
Mark attendance (all)	`POST /admin/events/{id}/attendance` `{ "records": [ { "registration_id": 1, "attendance_status": "attended" }, … ] }`

- Event date input: use `<input type="datetime-local">` and send `new Date(value).toISOString()`.
- **Attendance can only be recorded after the event date**; before that the API answers `409 ATTENDANCE_NOT_ALLOWED`. Hide/disable the attendance controls until `now >= event_date`. Task assignment works at any time (except cancelled events).
- Using the **bulk** attendance endpoint (`POST …/attendance`) also marks a still-`published` event as `completed`. The single-volunteer PATCH does not change the event status.
- Past events flip from `published` to `completed` automatically (within a minute of the start time).

6.7 Admin — volunteers

- List/search: `GET /admin/volunteers?status=pending&q=asha&page=1` (search matches name, email, phone). Typical tabs: **Pending**, **Approved**, **Deactivated**.
- Detail: `GET /admin/volunteers/{id}` → profile + `assignment_history: [RegistrationOut]`.
- **Approve** `POST /admin/volunteers/{id}/approve` — works from `pending` and also **re-activates** a `deactivated` volunteer.
- **Deactivate / reject** `POST /admin/volunteers/{id}/deactivate` — from `pending` (= reject) or `approved`.
- Both return the updated `VolunteerOut`; `409 INVALID_STATE` if it's already in that state.

6.8 Admin — donors & donations

- Search: `GET /admin/donors?q=priya&page=1` (matches name, e-mail, phone). Rows include `total_donated` and `donation_count`.
- Detail: `GET /admin/donors/{id}` → donor + `by_campaign: [ { campaign_id, campaign_title, total_amount, donation_count, last_donation_date } ]` + all `donations`.
- **Duplicate flags** `GET /admin/donors/duplicates` → groups of donor records sharing one e-mail: `json [ { "email": "donor1@example.com", "donors": [ DonorSummary, DonorSummary, DonorSummary ] } ]` UI: one card per group; let the admin pick the **primary** record (pre-select the one with `is_registered: true`), then **Merge**: `POST /admin/donors/merge` `{ "primary_donor_id": 1, "duplicate_donor_ids": [5,`

`6] }` → returns the merged `DonorSummary`. Donations move to the primary record; duplicates are deleted. A registered donor (login account) can never be on the duplicates side → `409 CANNOT_MERGE_REGISTERED`.

- All donations: `GET /admin/donations?campaign_id=&donor_id=&status=&page=` (read-only overview).

6.9 Admin — logs & staff

- `GET /admin/audit-log?entity_type=fund_utilization&action=&page=` — who did what and when. `entity_type` values include `campaign`, `event`, `volunteer`, `donor`, `fund_utilization`, `event_registration`. Each row: `{ id, user_id, action, entity_type, entity_id, details, created_at }`.
- `GET /admin/email-log?page=` — every e-mail attempt: `status` is `sent`, `skipped` (no SMTP configured) or `failed`. Helpful during demos.
- `POST /admin/admins` `{ name, email, password }` → creates another admin.

---

7. UI state rules (status → what to show)

Campaign (`status`)

Status	Public sees it?	Admin buttons	Donations?
`draft`	no (404)	Edit (all fields), **Publish**	no
`active`	yes	Edit **description + end_date only**, **Close early**	yes (**Donate** button)
`closed`	detail only (not in list)	none (read-only). Fund utilization still allowed	no — hide Donate

Donation (`status`)

Status	Meaning	UI
`pending`	created, not paid yet	confirm screen
`completed`	paid, counted in campaign total	success + receipt
`failed`	payment declined, or cancelled	failure screen → Retry (creates a new `pending`)

Event sign-up button (volunteer)

Evaluate top to bottom: 1. Not logged in → **"Log in to sign up"** 2. Logged in but not `role === "volunteer"` → hide the button 3. `volunteer_status === "pending"` → disabled, "Awaiting approval" 4. `volunteer_status === "deactivated"` → disabled, "Account deactivated" 5. `event.my_registration` not null → **"Withdraw"** (and show "You're signed up") 6. `event.is_full` → disabled, "Event full" 7. otherwise → **"Sign up"**

(The server enforces every one of these; the checks above just avoid pointless error toasts. If two people race for the last slot you will still get `409 EVENT_FULL` — handle it: toast + refetch.)

Volunteer status banner

`pending` → info banner "awaiting approval"; `approved` → none; `deactivated` → warning banner.

Event (`status`) in admin

Status	Admin buttons
`draft`	Edit, **Publish**, Cancel
`published`	Edit, Cancel, roster/tasks; attendance after the date
`completed`	roster, attendance (view/edit)
`cancelled`	none (read-only)

---

8. Downloads: PDFs and CSV

Browsers can't attach an `Authorization` header to a plain `<a href>`. Two cases:

1. Guest receipt (no login) — the donation token works as a query parameter, so a plain link is fine:

```
<a href={`${API}/donations/${id}/receipt.pdf?token=${accessToken}`} target="_blank"
rel="noreferrer">Download receipt (PDF)</a>
```

2. Anything that needs a login (donor's own receipt, admin exports) — fetch with the header, then save the blob using `download()` from §4.1:

```
await download(`/donations/${id}/receipt.pdf`, undefined, "receipt.pdf");                // logged-in donor
await download(`/admin/campaigns/${cid}/fund-report/export`, { format: "csv" }, "report.csv"); // admin
```

The server sends `Content-Disposition` with a proper filename and CORS exposes it, so `download()` names the file correctly (e.g. `RCP-20261014-000009.pdf`, `fund-report-campaign-1.csv`).

To show a PDF inline (preview iframe) use `const url = URL.createObjectURL(await (await api(path, {raw:true})).blob())`.

---

9. Cache refresh cheat-sheet

After a successful action, refetch these (TanStack Query keys are suggestions):

Action	Refetch
Donation confirmed (completed)	`campaign:{id}`, `campaigns` list, `donor dashboard`, `donor donations`, `admin dashboard`

Action	Refetch
Admin publishes/closes/edits campaign	`admin campaigns`, `campaign:{id}`, `campaigns` (public)
Fund utilization recorded	`utilizations:{id}`, `fund-report:{id}`, `admin dashboard`
Volunteer signs up / withdraws	`event:{id}` (counts + `my_registration`), `events`, `volunteer tasks`, `volunteer dashboard`
Admin publishes/edits/cancels event	`admin events`, `event:{id}`, `events`
Admin sets task / attendance	`roster:{id}`, `event:{id}`
Admin approves/deactivates volunteer	`admin volunteers`, `admin dashboard`
Donor merge	`admin donors`, `duplicates`
Profile edited	`me` (volunteer/donor), `/auth/me` (name changes)

Campaign/event counts change as other people act. For the campaign progress bar on a detail page you may poll `GET /campaigns/{id}` every 20–30 s (optional).

---

10. Error codes catalog

`code` values you may receive (HTTP status in parentheses). Unknown codes should fall back to showing `detail`.

Code	HTTP	When	Suggested UI
`VALIDATION_ERROR`	422	Any malformed/invalid input; see `errors[]`	Inline field messages
`NOT_AUTHENTICATED`	401	No token on a protected route (or donation flow without token/login)	Go to login
`INVALID_TOKEN`	401	Expired / bad token	Client auto-clears (§4.1); show login
`INVALID_CREDENTIALS`	401	Wrong e-mail or password	Form-level error
`ACCOUNT_DISABLED`	403	User account disabled	Message
`FORBIDDEN`	403	Wrong role / not your data	"Not allowed"
`NOT_FOUND`	404	Unknown id / hidden draft	404 state
`EMAIL_ALREADY_REGISTERED`	409	Register with used e-mail (field `email`)	Inline
`PHONE_ALREADY_REGISTERED`	409	Phone already used (field `phone_number`)	Inline
`ALREADY_REGISTERED`	409	Race on registration	Message

Code	HTTP	When	Suggested UI
`WRONG_PASSWORD`	400	Change password: current is wrong	Inline `current_password`
`INVALID_STATE`	409	Action not valid in current status (publish a non-draft, approve an approved volunteer, …)	Toast + refetch
`INVALID_DATES`	400	Campaign end date not after start / already passed	Inline `end_date`
`CAMPAIGN_CLOSED`	409	Edit a closed campaign	Toast
`CAMPAIGN_FIELD_LOCKED`	409	Edited a locked field of an active campaign	Toast; disable those inputs
`CAMPAIGN_NOT_ACTIVE`	409	Donate to / confirm for a non-active campaign	"Campaign is closed"
`CAMPAIGN_NOT_STARTED`	409	Record funds on a draft campaign	Toast
`DONOR_DETAILS_REQUIRED`	400	Guest donation without `donor`	Inline `donor`
`DONATION_LIMIT_EXCEEDED`	400	Amount above max (field `amount`)	Inline
`DONATION_NOT_PENDING`	409	Confirm/cancel a donation that is no longer pending (double click)	Refetch the donation
`DONATION_NOT_FAILED`	409	Retry a donation that didn't fail	Refetch
`RECEIPT_NOT_AVAILABLE`	409	Receipt for a non-completed donation	Hide receipt
`UTILIZATION_EXCEEDS_RAISED`	400	Spent total would exceed raised (field `amount`, message has the max)	Inline
`VOLUNTEER_NOT_APPROVED`	403	Pending/deactivated volunteer tries to sign up	Show status banner
`EVENT_NOT_OPEN`	409	Event is draft/cancelled/completed	Refetch
`EVENT_PASSED`	409	Event already started (sign-up/withdraw)	Refetch
`EVENT_FULL`	409	No slots left	"Event full" + refetch
`EVENT_DATE_IN_PAST`	400	Create/publish/edit with a past date (field `event_date`)	Inline
`SLOTS_BELOW_REGISTERED`	400	Lowered `volunteers_required` below current sign-ups	Inline
`ATTENDANCE_NOT_ALLOWED`	409	Attendance before the event date / on cancelled event	Disable control
`INVALID_MERGE`	400	Primary listed as duplicate	Inline
`CANNOT_MERGE_REGISTERED`	409	Tried to merge away a registered donor	Message

Code	HTTP	When	Suggested UI
`HTTP_xxx`	4xx	Anything else, e.g. wrong verb gives `METHOD_NOT_ALLOWED` (405)	Generic
`INTERNAL_ERROR`	500	Server bug	Generic message

---

11. Complete endpoint reference

Auth column: **—** none · **opt** token optional · **any** any logged-in user · **donor** / **volunteer** / **admin** role required. All paths are relative to `/api/v1`. Lists marked *paged* return the pagination envelope.

Auth

Method	Path	Auth	Body	Returns
POST	`/auth/register/volunteer`	—	`name, email, password, phone_number, areas_of_interest[], availability?`	`201` AuthResponse
POST	`/auth/register/donor`	—	`name, email, password, phone_number?`	`201` AuthResponse
POST	`/auth/login`	—	`email, password`	AuthResponse
POST	`/auth/token`	—	form `username, password` (Swagger only)	`{access_token, token_type, expires_in}`
GET	`/auth/me`	any	—	UserOut
POST	`/auth/change-password`	any	`current_password, new_password`	`{detail}`

Public

Method	Path	Auth	Query	Returns
GET	`/campaigns`	—	`page, page_size, category, q, sort`	*paged* CampaignOut (active only)
GET	`/campaigns/categories`	—	—	`string[]`
GET	`/campaigns/{id}`	opt	—	CampaignOut (draft → 404 unless admin)
GET	`/events`	opt	`page, page_size, q, include_past`	*paged* EventOut
GET	`/events/{id}`	opt	—	EventOut (draft → 404 unless admin)
GET	`/meta/config`	—	—	enums, limits, suggestion lists
GET	`/meta/faq`	—	—	`[{topic, question, answer}]`
GET	`/health` *(no `/api/v1` prefix)*	—	—	`{"status":"ok"}`

Donations (guest or donor)

Header `X-Donation-Token: <access_token>` (or `?token=`) authorises the donation; a logged-in donor may use their JWT instead; admins can read (`GET`, receipt) but not act.

Method	Path	Auth	Body	Returns
POST	`/donations`	opt	`campaign_id, amount, payment_method, donor?, pan_tax_id?`	`201` DonationInitiated
GET	`/donations/{id}`	token / donor-owner / admin	—	DonationOut
POST	`/donations/{id}/confirm`	token / donor-owner	optional `{simulate_outcome}`	DonationOut
POST	`/donations/{id}/cancel`	token / donor-owner	—	DonationOut
POST	`/donations/{id}/retry`	token / donor-owner	—	`201` DonationInitiated (new id + new token)
GET	`/donations/{id}/receipt`	token / owner / admin	—	ReceiptOut
GET	`/donations/{id}/receipt.pdf`	token / owner / admin	—	PDF file

Volunteer

Method	Path	Auth	Body	Returns
POST	`/events/{id}/signup`	volunteer (approved)	—	`201` RegistrationOut
DELETE	`/events/{id}/signup`	volunteer	—	`{detail}`
GET	`/volunteers/me`	volunteer	—	VolunteerOut
PATCH	`/volunteers/me`	volunteer	`name?, phone_number?, areas_of_interest?, availability?`	VolunteerOut
GET	`/volunteers/me/tasks`	volunteer	—	`RegistrationOut[]` (upcoming)
GET	`/volunteers/me/history`	volunteer	—	`RegistrationOut[]` (all)
GET	`/volunteers/me/dashboard`	volunteer	—	VolunteerDashboard

Donor

Method	Path	Auth	Body / Query	Returns
GET	`/donors/me`	donor	—	DonorSummary
PATCH	`/donors/me`	donor	`name?, phone_number?`	DonorOut
GET	`/donors/me/donations`	donor	`page, page_size, status?`	*paged* DonationOut
GET	`/donors/me/dashboard`	donor	—	DonorDashboard

Admin — campaigns & funds (all `admin`)

Method	Path	Body / Query	Returns
GET	`/admin/campaigns`	`page, page_size, status?, q?`	*paged* CampaignOut
POST	`/admin/campaigns`	`title, description, category, goal_amount, start_date, end_date`	`201` CampaignOut (draft)
GET	`/admin/campaigns/{id}`	—	CampaignOut
PATCH	`/admin/campaigns/{id}`	any of the create fields	CampaignOut
POST	`/admin/campaigns/{id}/publish`	—	CampaignOut
POST	`/admin/campaigns/{id}/close`	—	CampaignOut
POST	`/admin/campaigns/{id}/utilizations`	`category, amount, description, reference_number, utilization_date?`	`201` UtilizationOut
GET	`/admin/campaigns/{id}/utilizations`	—	`UtilizationOut[]`
GET	`/admin/campaigns/{id}/fund-report`	—	FundReport
GET	`/admin/campaigns/{id}/fund-report/export`	`format=pdf` or `csv`	file download
GET	`/admin/dashboard`	—	AdminDashboard

Admin — events (all `admin`)

Method	Path	Body / Query	Returns
GET	`/admin/events`	`page, page_size, status?, q?`	*paged* EventOut
POST	`/admin/events`	`title, description, event_date, location, volunteers_required`	`201` EventOut (draft)
GET	`/admin/events/{id}`	—	EventOut
PATCH	`/admin/events/{id}`	any create field	EventOut
POST	`/admin/events/{id}/publish`	—	EventOut

Method	Path	Body / Query	Returns
POST	`/admin/events/{id}/cancel`	—	EventOut
GET	`/admin/events/{id}/registrations`	—	`RosterEntry[]`
PATCH	`/admin/events/{id}/registrations/{regId}`	`task?, attendance_status?` (≥ 1)	RosterEntry
POST	`/admin/events/{id}/attendance`	`records: [{registration_id, attendance_status}]`	`RosterEntry[]`

Admin — people & logs (all `admin`)

Method	Path	Body / Query	Returns
GET	`/admin/volunteers`	`page, page_size, status?, q?`	*paged* VolunteerOut
GET	`/admin/volunteers/{id}`	—	VolunteerAdminDetail
POST	`/admin/volunteers/{id}/approve`	—	VolunteerOut
POST	`/admin/volunteers/{id}/deactivate`	—	VolunteerOut
GET	`/admin/donors`	`page, page_size, q?`	*paged* DonorSummary
GET	`/admin/donors/duplicates`	—	`DuplicateGroup[]`
POST	`/admin/donors/merge`	`primary_donor_id, duplicate_donor_ids[]`	DonorSummary
GET	`/admin/donors/{id}`	—	DonorDetail
GET	`/admin/donations`	`page, page_size, campaign_id?, donor_id?, status?`	*paged* DonationOut
GET	`/admin/audit-log`	`page, page_size, entity_type?, action?`	*paged* AuditLogOut
GET	`/admin/email-log`	`page, page_size`	*paged* EmailLogOut
POST	`/admin/admins`	`name, email, password`	`201` UserOut

12. Data shapes (TypeScript-style)

```
type Money = string;                 // "1500.00"
type ISODate = string;               // "2026-10-23"
type ISODateTime = string;           // "2026-10-19T05:23:57.655088Z"

interface Page<T> { items: T[]; total: number; page: number; page_size: number; pages: number }

interface UserOut { id: number; email: string; full_name: string; role: "admin"|"volunteer"|"donor";
is_active: boolean;
                    volunteer_id: number|null; volunteer_status:
"pending"|"approved"|"deactivated"|null; donor_id: number|null }
interface AuthResponse { access_token: string; token_type: "bearer"; expires_in: number; user:
UserOut }

interface CampaignOut { campaign_id: number; title: string; description: string; category: string;
  goal_amount: Money; amount_raised: Money; amount_remaining: Money; progress_percent: number;   //
0-100, 1 decimal
  start_date: ISODate; end_date: ISODate; status: "draft"|"active"|"closed";
  closed_reason: "goal_reached"|"end_date_passed"|"manual"|null;
  created_at: ISODateTime; published_at: ISODateTime|null; closed_at: ISODateTime|null }

interface DonationOut { donation_id: number; campaign_id: number; campaign_title: string; donor_id:
number; donor_name: string;
  amount: Money; payment_method: "card"|"upi"|"net_banking"; status: "pending"|"completed"|"failed";
  receipt_number: string|null; donation_date: ISODateTime; completed_at: ISODateTime|null;
  failure_reason: string|null; retry_of_id: number|null }
interface DonationInitiated { donation: DonationOut; access_token: string;
  confirmation: { campaign_title: string; amount: Money; payment_method: string; donor_name: string
} }
interface ReceiptOut { receipt_number: string; ngo_name: string; donor_name: string; donor_email:
string; pan_tax_id: string|null;
  amount: Money; currency: "INR"; campaign_title: string; payment_method: string; payment_reference:
string|null; donation_date: ISODateTime }

interface EventOut { event_id: number; title: string; description: string; event_date: ISODateTime;
location: string;
  volunteers_required: number; status: "draft"|"published"|"cancelled"|"completed";
  registered_count: number; slots_remaining: number; is_full: boolean;
  my_registration: { registration_id: number; task: string|null; attendance_status:
"registered"|"attended"|"no_show" } | null }
interface RegistrationOut { registration_id: number; task: string|null; attendance_status:
"registered"|"attended"|"no_show";
  registered_at: ISODateTime; event: { event_id: number; title: string; event_date: ISODateTime;
location: string; status: string } }
interface RosterEntry { registration_id: number; task: string|null; attendance_status: string;
registered_at: ISODateTime;
  volunteer: { volunteer_id: number; name: string; email: string; phone_number: string } }

interface VolunteerOut { volunteer_id: number; name: string; email: string; phone_number: string;
  areas_of_interest: string[]; availability: string|null; status:
"pending"|"approved"|"deactivated"; created_at: ISODateTime }
interface VolunteerAdminDetail extends VolunteerOut { assignment_history: RegistrationOut[] }
interface VolunteerDashboard { volunteer: VolunteerOut; upcoming_assignments: RegistrationOut[];
```

```
events_attended: number; open_events: EventOut[] }

interface DonorSummary { donor_id: number; name: string; email: string; phone_number: string|null;
is_registered: boolean;
  created_at: ISODateTime; total_donated: Money; donation_count: number }
interface DonorDetail extends DonorSummary { by_campaign: { campaign_id: number; campaign_title:
string; total_amount: Money; donation_count: number; last_donation_date: ISODateTime }[]; donations:
DonationOut[] }
interface DuplicateGroup { email: string; donors: DonorSummary[] }
interface DonorDashboard { donor: DonorSummary; recent_donations: DonationOut[]; featured_campaigns:
CampaignOut[] }

interface UtilizationOut { utilization_id: number; campaign_id: number; category: string; amount:
Money; description: string;
  reference_number: string; utilization_date: ISODate; recorded_by: number; recorded_by_name:
string; recorded_date: ISODateTime }
interface FundReport { campaign_id: number; campaign_title: string; campaign_status: string;
  total_raised: Money; total_utilized: Money; remaining_balance: Money;
  breakdown: { category: string; amount: Money; entries: number; percent_of_utilized: number }[];
  entries: UtilizationOut[]; generated_at: ISODateTime }
interface AdminDashboard { total_raised: Money; total_utilized: Money; total_remaining: Money;
  campaigns: { draft: number; active: number; closed: number }; total_donors: number;
  pending_volunteers: number; approved_volunteers: number; upcoming_events: number;
recent_donations: DonationOut[] }

interface ApiErrorBody { detail: string; code: string; errors?: { field: string|null; message:
string }[] }
```

13. Demo & testing tips

- **Force a payment failure/success** (so you can build and demo the failure screen): `POST /donations/{id}/confirm` with body `{ "simulate_outcome": "failure" }` (or `"success"`). Works while `GET /meta/config → payment_simulation_allows_forced_outcome` is `true`. In production builds the backend can disable it (`PAYMENT_ALLOW_FORCED_OUTCOME=false`) and then it is ignored. Also, `PAYMENT_FAILURE_RATE=0.3` in `backend/.env` makes 30 % of payments fail randomly. Suggested: a small hidden "Demo mode" toggle in the donation confirm screen that sends `simulate_outcome`.
- **No real money moves.** Payments are simulated (SRS CO-4). The PDF receipt says so.
- **See e-mails without a mail server:** open `/admin/email-log` (status `skipped` = would have been sent). To actually send, fill `SMTP_*` in `backend/.env`.
- **Swagger "Authorize"**: click it in `/docs` and enter `admin@ngo.org` / `Admin@12345` to try admin endpoints in the browser.
- **Reset the data:** stop the server, delete `backend/vdms.db*`, restart, re-run the seed.
- **Trigger auto-close of a campaign quickly:** create a campaign with a small `goal_amount` (e.g. `"100.00"`), publish it, donate `100.00` → it flips to `closed` with `closed_reason: "goal_reached"` immediately.
- **Test attendance:** attendance only works after the event date, which is always in the future for new events. To demo it, edit `event_date` directly in the DB (SQLite) or ask the backend dev to add a time-travel flag.

- **Run the backend tests:** `pip install -r requirements-dev.txt` then `pytest` in `backend/` (56 tests).

---

14. Design decisions & limitations

Things you might trip over — all intentional for Release 1.0:

1. **One role per account / e-mail.** A person who is both donor and volunteer needs two e-mails.
2. **E-mail is the login id and can't be changed** from the profile screens (name, phone, availability, interests can).
3. **No refresh tokens / logout endpoint.** Token lifetime is 8 h; logout = delete the token client-side.
4. **Guest donations are linked by e-mail + name.** A guest never attaches to a registered account that shares their e-mail; admins see such pairs under *Duplicates* and can merge.
5. **"Cancel" a donation = status** `failed` with `failure_reason: "Cancelled by donor"` (the SRS allows only pending/completed/failed).
6. **A donation may push a campaign over its goal.** The last donation is accepted in full; the campaign then closes (`goal_reached`).
7. **Publishing makes a campaign active immediately**, regardless of `start_date` (which is informational). A campaign can't be published if its `end_date` already passed.
8. **Campaigns close automatically** at the end of the `end_date` day (Asia/Kolkata) or when the goal is reached. Reads trigger this immediately; a background job also runs every 60 s.
9. **Fund reports are admin-only** in this release (SRS "authorized viewer" = admin).
10. **Volunteer deactivation** blocks *new* sign-ups only; the volunteer can still log in and see their data. Existing sign-ups are kept.
11. **Event cancellation** keeps the sign-up records (for history) and e-mails the volunteers.
12. **Receipt PDFs use** `Rs.` instead of the ₹ glyph (built-in PDF font limitation); the on-screen receipt can use ₹.
13. **Not in this release:** real payment gateway, SMS/WhatsApp, recurring donations, multi-branch (SRS "future releases").
14. **HTTPS (SRS CI-3/SE-1)** is a deployment concern: run behind a TLS-terminating proxy (nginx/Caddy/cloud). Locally plain HTTP is used.

---

15. SRS traceability

Which endpoint implements which SRS requirement (useful for the viva / demo).

SRS requirement	Where
Volunteer.Register (+ `.Validate`)	`POST /auth/register/volunteer`; duplicates → `409 EMAIL_/PHONE_ALREADY_REGISTERED`
Volunteer.Approve / BR-6	`POST /admin/volunteers/{id}/approve`; unapproved sign-up → `403 VOLUNTEER_NOT_APPROVED`
Volunteer.Profile.Edit / .View	`PATCH /volunteers/me` / `GET /admin/volunteers/{id}`
Volunteer.Deactivate	`POST /admin/volunteers/{id}/deactivate`

SRS requirement	Where
Volunteer.Task.View	`GET /volunteers/me/tasks`
Campaign.Create (+ `.Validate`)	`POST /admin/campaigns` (422 on bad dates / goal ≤ 0)
Campaign.Publish	`POST /admin/campaigns/{id}/publish`
Campaign.View.Active	`GET /campaigns`
Campaign.Progress.Update	automatic on donation confirm (`amount_raised`)
Campaign.Close.Auto / .Manual	automatic (goal / end date) / `POST /admin/campaigns/{id}/close`
Campaign.Edit	`PATCH /admin/campaigns/{id}` (goal locked once active)
Donation.Initiate / .Amount.Validate	`POST /donations` (amount > 0, ≤ configurable max)
Donation.Guest	`donor` object + optional `pan_tax_id`
Donation.Payment.Select	`payment_method`: `card`, `upi`, `net_banking`
Donation.Confirm	`confirmation` summary → `POST /donations/{id}/confirm`
Donation.Process.Success / .Failure	`completed` (total updated) / `failed` (total untouched)
Donation.Receipt.Generate / .Email	`GET /donations/{id}/receipt(.pdf)`; background e-mail
Donation.History.View	`GET /donors/me/donations`
Donor.Record.Create	automatic on first donation
Donor.Record.Search / .View	`GET /admin/donors?q=` / `GET /admin/donors/{id}`
Donor.Profile.Edit	`PATCH /donors/me`
Donor.Merge.Flag	`GET /admin/donors/duplicates` (+ `POST /admin/donors/merge`)
Event.Create / .Publish	`POST /admin/events`, `POST /admin/events/{id}/publish`
Event.Signup / .Signup.Full	`POST /events/{id}/signup` (`409 EVENT_FULL`)
Event.Task.Assign	`PATCH /admin/events/{id}/registrations/{regId}`
Event.Attendance.Mark	same PATCH or `POST /admin/events/{id}/attendance`
Event.Cancel	`POST /admin/events/{id}/cancel` (+ e-mails)
Event.History.View	`GET /volunteers/me/history`
Fund.Utilization.Record / .Validate	`POST /admin/campaigns/{id}/utilizations` (BR-4 → `400 UTILIZATION_EXCEEDS_RAISED`)
Fund.Report.Campaign / .Export / .Dashboard	`GET …/fund-report`, `…/fund-report/export`, `GET /admin/dashboard`
SE-2 / SE-4 / BR-5	JWT login; every `/admin/**` route is admin-only; role checks on `/volunteers/me`, `/donors/me`
SE-3	bcrypt salted password hashes
SE-5	donors can only read their own donations
SE-6	`GET /admin/audit-log` (admin id + timestamp for fund entries)

SRS requirement	Where
CO-3	integer-paise storage, two-decimal strings
CO-4 / SI-2	simulated payment module behind an interface
2.6 (help/FAQ)	`GET /meta/faq`
UI-1..UI-4, Usability-1	frontend responsibilities (public browse, role dashboards, responsive, inline validation, ≤ 4-step donation)

---

16. Integration checklist

Setup - [ ] Backend runs (`/health` is ok), Swagger opens at `/docs` - [ ] Dev proxy or `CORS_ORIGINS` configured; no trailing slashes in any request - [ ] API client with token storage + `INVALID_TOKEN` retry-as-anonymous (§4.1) - [ ] `AuthProvider` restores the session via `GET /auth/me`; route guard per role; redirect by role after login

Public - [ ] Campaign list (filters, sort, pagination, progress bar from `progress_percent`) - [ ] Campaign detail with Donate button only for `active` - [ ] Donation flow: form → confirm → success/failure; token kept; retry creates a new donation; Pay button disabled while loading - [ ] Receipt page + PDF download (token link for guests, blob download for logged-in) - [ ] Events list/detail with the sign-up button state machine (§7); FAQ page

Auth & roles - [ ] Login / register volunteer / register donor with inline server errors (`errors[].field`) - [ ] Volunteer "awaiting approval" and "deactivated" banners - [ ] Donor: dashboard, history (receipt links only for completed), profile - [ ] Volunteer: dashboard, tasks, history, profile, sign up / withdraw

Admin - [ ] Dashboard numbers + "pending volunteers" call-to-action - [ ] Campaign CRUD with status-dependent buttons and locked fields when active - [ ] Fund utilization form (max = `remaining_balance`), report with breakdown chart, PDF/CSV download - [ ] Event CRUD, publish, cancel (with "N volunteers will be e-mailed"), roster, task assignment, attendance (disabled before the event date) - [ ] Volunteers approve/deactivate; donors search/detail; duplicates + merge; donations list; audit log; e-mail log; add admin

Polish - [ ] Money formatted with `Intl.NumberFormat("en-IN")`, dates shown as DD/MM/YYYY - [ ] 409 errors show `detail` in a toast and refetch the affected data - [ ] Responsive layouts (volunteers/donors are mostly on mobile — SRS UI-3)